

基于 OpenVela 的视障随行视觉辅助胸牌

1、信息表

项目	内容
作品名称	基于 OpenVela 的视障随行视觉辅助胸牌
队伍名称	不知道叫什么名字（队伍编号 441）
团队分工	邢晓亮（GitHub：xxluestc）：BSP 移植、驱动适配、构建烧录、实机验证、应用与报告整理
选题方向	新硬件平台适配

2、摘要

本项目面向具有独立出行能力的视障用户，开发一款基于 OpenVela 的免手持视觉辅助胸牌，用于在用户到达车站、教学楼、医院、商场等目标区域后，识别入口、门牌、站点标识和公共设施等视觉信息。项目在 BK7258 R1 上完成物理 CPU0 与 AP 侧 NuttX/OpenVela 启动、RPTUN/RPMsg、AP 双核 SMP、双屏 LCD、GC2145 摄像头、SD 和 Wi-Fi 数据链路，并将拍照识别应用接入 MiMo。r14 实机支持 KEY1 短接触发，串口记录了 21,099 字节 JPEG、中文识别正文及 stage=done status=0，双屏运行眼睛动画。后续将在现有视觉闭环上完善语音提问、语音播报及 LED/振动反馈，形成更自然的多通道辅助交互。

关键词：OpenVela；NuttX；BK7258；RPTUN；GC2145；MiMo；视觉辅助；无障碍

3、正文

3.1 绪论

3.1.1 项目背景与问题定义

本项目面向具有独立出行能力的视障用户。用户到达车站、教学楼、医院、商场或办事区域后，仍可能因为无法及时读取入口、门牌、楼层、站点名称和公共设施标识而难以确认最后一段路线。持续手持手机拍摄还会占用双手，影响通行和环境感知。视觉辅助胸牌固定在胸前，以按键或后续语音触发单次观察，把用户前方画面转成简短、可理解的环境描述。

围绕这一应用场景，本项目选择声网与博通集成的对话式 AI 开发套件 R1。项目首先完成 BK7258 的全 OpenVela BSP 适配，建立 CPU0、AP 和 CPU2 的启动、调度、跨核通

信与板级驱动基础，再用双屏、摄像头、Wi-Fi 和 MiMo 视觉问答验证系统可用性。当前 r14 已形成 KEY1 拍照、云端图片理解、中文结果返回和双屏状态展示的实机闭环。

3.1.2 技术难点

三颗物理 CPU 与两个 NuttX 层级容易混淆。物理 CPU0 负责系统首启和 AP 管理；物理 CPU1 是 AP NuttX 的逻辑 CPU0；物理 CPU2 是 AP NuttX SMP 的逻辑 CPU1。只看到 AP NSH 不能证明 CPU0 也是 openvela，只看到 /proc/cpuinfo 的 processor: 0 也不能证明 CPU2 已上线。

启动和内存边界必须同时正确。CPU0、AP、bootloader、共享区、PSRAM 和用户数据有固定 Flash/RAM 契约。入口地址、VTOR、MSP、链接脚本或镜像偏移任何一处不一致，都可能表现为无日志、AP 超时或 RPTUN 长期停在 CONNECTING。

芯片层与板级层必须分开。GPIO 寄存器、IRQ、SysTick 属于 BK7258 芯片层；按键、LED、屏幕复位、电源脚和 Camera 引脚属于 R1 板级映射。把板级常量写进通用 SoC 代码会破坏复用，也容易把相近开发板误当成同一硬件。

跨核外设服务不是普通函数调用。Wi-Fi、Flash 等能力需要明确控制器所在 CPU、RPMsg 服务、共享缓冲所有权和错误恢复；-107、-16 等应用错误往往只是前面 AP 或服务未就绪的连锁结果。

多媒体容易触发资源冲突。两块 160×160 RGB565 帧缓冲共需 102400 字节，一帧 640×480 VYUY 图像需 614400 字节，还要与网络、TLS 和 JPEG 缓冲共享 SRAM/PSRAM。

板级能力需要实机闭环。源码接入、配置生效、构建通过和开发板响应是不同层级，必须结合镜像哈希与串口日志逐层确认。

3.1.3 创新点与差异化

把 R1 从“原厂 CP + openvela AP”的混合验证推进为 CPU0 与 AP 均运行 NuttX/openvela、CPU2 进入 AP SMP 的全 OpenVela 路线，并以状态机和实机日志明确证明每一层。

将多核启动、RPTUN 通道、双屏显示和视觉识别的实机结果绑定到镜像与串口记录，便于复核。

将“源码 → 配置 → 构建 → ELF/bin → 合包 → Flash → CPU 启动 → bring-up → 设备节点/服务”做成可追溯流程，所有结论绑定提交、镜像哈希、串口日志和通过标准。

自建 bk7258-openvela-porting Skill，把 BSP 边界核对、R1 差异、构建同步、失败处理和证据门禁固化为可重复流程，减少反复猜接口和无证据修改。

3.2 系统方案设计

3.2.1 总体架构与数据流

本次实机系统采用以下拓扑：

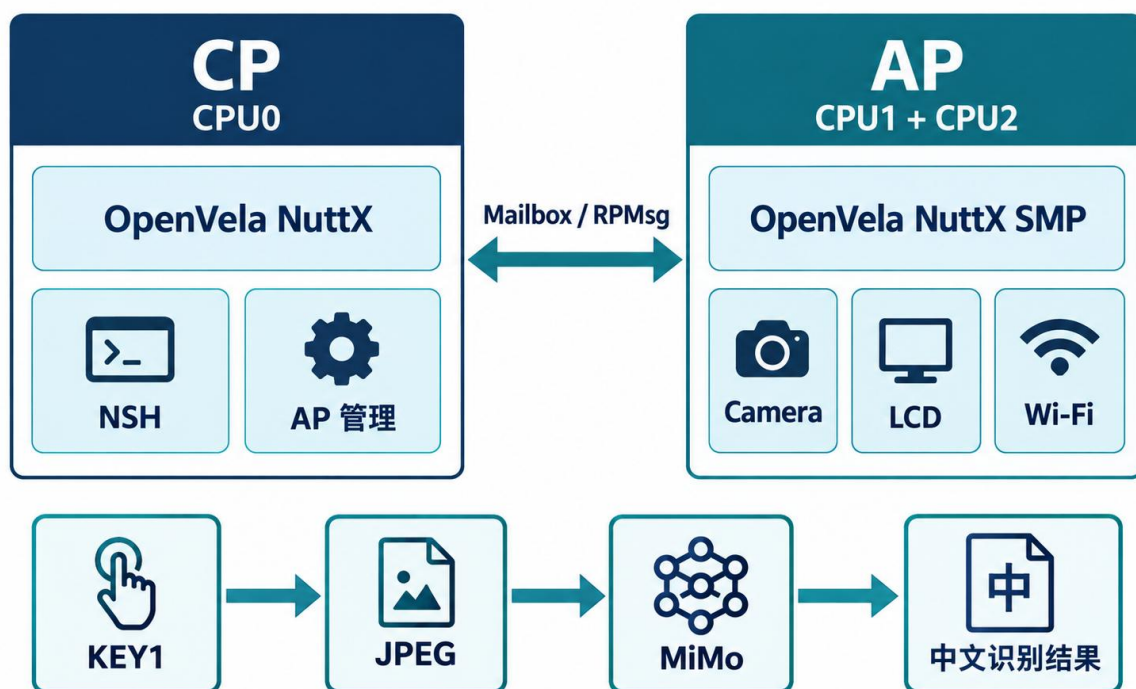


图 1 BK7258 R1 系统结构及视觉数据流

主要数据流分为四条：

显示：应用像素或状态 → framebuffer → GC9D01 lower-half → QSPI/SPI → 两块 160×160 屏幕。

视觉：GC2145 寄存器配置 → I2C 与 DVP → YUV buffer/DMA → PSRAM 帧缓冲 → JPEG → V4L2 /dev/video0 → 应用服务。

网络：应用请求 → NuttX 网络栈 → 跨核 Wi-Fi 服务 → Beken PHY/RF；DHCP、DNS、TCP/TLS 按顺序建立。

3.2.2 方案论证与选型

本项目选择 CPU0 与 AP 均运行 OpenVela/NuttX，AP 侧由 CPU1、CPU2 组成 SMP 系统。CP 负责启动管理、串口命令和跨核请求；AP 承担 Camera、LCD、Wi-Fi 和 MiMo 请求。两侧通过 Mailbox、共享内存及 RPTUN/RPMsg 通信，这一分工使视觉交互与硬件驱动保持清晰边界。

R1 基于 BK7258，集成摄像头、Wi-Fi、麦克风、扬声器、双圆屏、按键、LED 和振动器，硬件能力与免手持视觉辅助胸牌高度匹配。当前版本以摄像头采集、Wi-Fi 通信、MiMo 图片理解和双屏反馈完成核心验证；麦克风、扬声器、LED 与振动器为后续语音输入、语音播报和多通道提醒提供硬件基础。端侧负责系统启动、驱动、采集、缓存和设备控制，云端负责图片理解与自然语言回答。

3.2.3 关键模块设计

CPU0 启动模块：初始化 MSP/VTOR、FPU、时钟、低级串口、IRQ、SysTick 和堆，进入 nx_start()，再由 NSH 和 AP 管理应用提供可观测入口。

AP 管理模块：用明确状态记录 AP 启动、心跳、错误和 generation；失败时保留第一处错误，不让后续网络错误覆盖根因。

跨核通信模块：通过 Mailbox/doorbell 触发事件，共享内存承载数据，RPTUN/RPMsg 管理通道生命周期；状态至少区分 CONNECTING 和 CONNECTED。

SMP 模块：AP 的逻辑 CPU0 拉起物理 CPU2，验证第二调度 CPU、IPI 和双向回调；用 online 位图和 PASSED 状态，而不是只看启动字符串。

板级 bring-up: 将通用 BK7258 驱动与 R1 的引脚、电源、总线和设备映射组合, 按依赖顺序注册字符设备、块设备、网络和多媒体设备。

应用服务层: vision_badge 通过 camera、vision、feedback 和 RPC 接口组织 workflow。当前由 KEY1 或 NSH 命令触发摄像头采集, AP 将 JPEG 经 Wi-Fi 发送至 MiMo, 并把中文结果返回 CP; 双屏眼睛动画提供直观状态反馈。后续将在保持现有接口边界的基础上接入麦克风语音提问、回答语音播报以及 LED/振动提醒。

3.3 核心算法与技术原理

3.3.1 启动、调度与系统节拍

复位后, 链接脚本给出的向量表、初始 MSP 和程序入口必须与 Flash 中的实际镜像地址一致。bk7258_start.c 完成早期架构初始化, bk7258_lowputc.c 提供调度器启动前的串口输出, bk7258_irq.c 建立中断, bk7258_timerisr.c 配置系统节拍, 最后进入 NuttX 调度。

SysTick 不只是“一个定时器”。NuttX 的延时、超时和时间片都依赖持续递增的系统 tick。项目早期出现 NSH 已能输入、但 sleep 不返回的现象, 说明 UART 和命令解释器已运行, 而系统节拍链未闭合。修正 SysTick 后, sleep 和定时任务恢复, 这一案例用于验证“先找第一处失效的公共依赖”。

3.3.2 多核启动、SMP 与跨核通信

CPU0 先启动并建立共享区, 再释放 AP。AP 的首核完成运行时重定位、堆和调度初始化后, 与 CPU0 建立 RPTUN; 随后 AP 首核通过启动入口、栈和 IPI 机制拉起物理 CPU2。三类状态含义不同:

AP READY: AP 核心已经运行并能响应管理侧;

RPTUN CONNECTED: 两端共享内存和通知通道已经完成握手;

CPU2 SCHEDULER_ONLINE 与 AP SMP PASSED: AP 第二调度 CPU 已上线, 并完成双向调度/回调查证。

本队早期实机日志中三者同时出现且 heartbeat 递增, 证明启动和跨核基本链可工作; r5 自检再次记录 AP READY、RPTUN CONNECTED、CPU2 SCHEDULER_ONLINE 与 AP SMP PASSED。

3.3.3 内存与资源约束

项目使用链接脚本和构建后检查共同约束 Flash 与 RAM。r14 分区中, bootloader 位于 0x00000000, CPU0 应用位于 0x00011000, AP 位于 0x00143000, pair 位于 0x002ca000; EasyFlash、RF 校准和用户数据分区不被本次局部更新覆盖。AP 运行区与 CPU0/SWAP 边界同样需要检查, 不能只看 bin 文件大小。

大缓冲优先放入 PSRAM, DMA 缓冲还要满足硬件可访问范围、对齐和缓存一致性。两块显示帧缓冲、VGA 图像、JPEG 工作区和 TLS 内存不能各自按峰值无限申请, 终版以实际 map、free 和重复开关测试决定预算。

3.3.4 openvela 设备模型与多媒体机制

NuttX 通过 upper-half/lower-half 把应用接口与硬件实现分离。板级 bringup 调用注册函数后, 应用经文件描述符和 ioctl 使用 /dev/fb*、/dev/video*、/dev/mmc* 等设备, 而不是直接调用板级寄存器函数。这样可把 GC2145、GC9D01 等 R1 差异留在 lower-half 和 board 层, 并让应用服务保持稳定。

r13/r14 的全 OpenVela BSP 实机日志记录了 Camera JPEG、SD 数据面和双屏眼睛线程。GC2145 通过 DVP/DMA 将图像送入缓冲并完成 JPEG 编码; 两块圆屏由独立显示链路驱动。

3.3.5 AI 能力边界

AI 能力采用云端 MiMo 图片理解。r14 实机已完成 Camera JPEG、Wi-Fi/DHCP、HTTPS、图片上传、JSON 解析、跨核返回和中文结果打印; KEY1 可触发这一链路。API Key 从设备侧配置加载, 不写入镜像和普通日志。

3.4 系统实现

3.4.1 软件与固件结构

当前队伍仓按职责分为：chips/bk7258 芯片层、boards/bk7258 板级层、nuttx 覆盖层、tools/bk7258 构建工具、app/bsp_diag 诊断命令、app/vision_badge 视觉应用、移植审查 Skill、实机证据和 AI Coding 日志。

本队将 CP/AP 配置、R1 板级绑定、视觉应用、诊断工具与构建规则组织在同一赛事仓库。固件依赖由 manifest 固定；各组件保留相应许可证声明。

3.4.2 从源码到实机的完整流程

完整流程为：Kconfig/defconfig 选择功能，编译生成 CPU0/AP ELF，使用 map/readelf/nm 检查入口和边界，生成 CPU0/AP bin，按分区表与 bootloader 合包并生成 SHA256，由参赛者将明确 Flash 范围写入开发板，最后依次验收 CPU0、AP、RPTUN、CPU2 SMP、board bring-up、NSH 和设备节点。

烧录地址不是从文件名推测，而是由分区表、链接地址和打包清单共同确定。项目曾用局部写入只更新 CPU0/AP，避开 boot、配对、EasyFlash、RF 校准和用户数据区域；每个候选镜像都应记录输入文件、地址、长度、工具版本、日志和 SHA256，并保留已验证的回退镜像。

3.4.3 芯片层与板级适配

bk7258_start.c：复位入口与进入 NuttX 前的架构初始化；

bk7258_lowputc.c、bk7258_serial.c：早期 UART 和 NuttX 串口设备；

bk7258_irq.c：中断控制器初始化与 IRQ 分发；

bk7258_timerisr.c：系统节拍；

bk7258_allocateheap.c：堆边界；

bk7258_gpio.c：与 SoC 相关的 GPIO 操作；

boards/bk7258/aidk_ai_toy/src/：R1 早期板级初始化、驱动注册与设备 bring-up；

bk7258_buttons.c、bk7258_userleds.c：R1 引脚映射与 NuttX board API。

GPIO 实测曾发现初始 LED 引脚映射错误：KEY1 和 KEY3 只能点亮一颗绿灯。回到 R1 原理图和板级映射修正后，KEY1/KEY2/KEY3 分别产生掩码 0x01/0x02/0x04，并对应红灯、绿灯和双灯。这说明不同 board 的引脚必须留在板级，不能通过猜测修改通用 GPIO 驱动。

3.4.4 应用和交互实现

vision_badge 把应用拆为 camera、vision、feedback、workflow 和 RPC 服务。CP 的 NSH 命令或 KEY1 短接通过 RPMsg 触发 AP 采图，AP 使用 V4L2 获取 JPEG、通过 HTTPS 请求 MiMo 并把有界中文结果返回 CP。r14 的 KEY1 路径在 CP 串口打印中文正文。

本次人机交互以 KEY1 短接拍照识别和双屏眼睛动画为主。r13 启动日志记录 AIDK EYE START vendor dual-eye animation 和眼睛任务调度；r14 沿用同一 AP 镜像，硬件照片记录双屏点亮，操作者观察到眨眼。

3.4.5 自建 Skill

自建 Skill 位于 .agents/skills/bk7258-openvela-porting/，不是一段泛化提示词，而是面向本项目的移植流程：

检查 R1 原理图/BOM、板级配置和关键外设映射；

核对 CP/AP 配置、分区与构建产物；

用审计脚本检查项目结构、配置和证据；

以“源码存在 → 配置生效 → 构建通过 → 镜像正确 → R1 实机通过”为证据门禁；对失败记录第一处异常、层级、回退点和下一实验。

仓库现有 Python 回归测试用于检查 Skill 审计规则。自建 Skill 是开发流程工具，不属于设备运行时 Agent Skill；本次参赛方向为新硬件平台适配。

3.5 系统测试与结果分析（务必提供量化数据）

3.5.1 测试环境

硬件为 BK7258 R1/AIDK AI Toy，串口 COM12，115200 8N1。固件为 r14，CP/AP 全 OpenVela，AP SMP=2。构建在 Ubuntu VM 通过 BK7258 direct build 完成，镜像哈希见 evidence/final/vision-badge-r14-SHA256SUMS.txt。MiMo 模型为 mimo-v2.5。实机证据集中在 evidence/final/，包括构建清单、烧录日志、KEY1 和命令触发的查询日志。

3.5.2 功能测试结果

CP 到 AP RPC 返回 AP endpoint ready。r13 同 AP 镜像启动日志记录 Wi-Fi DHCP ACK、Camera 数据面通过、SD 数据面通过和原厂样式眼睛任务启动。r14 的 KEY1 实测输出 VISION BADGE KEY1 short press; starting capture，采集 21,099 字节 JPEG，返回 81 字节中文结果，最终为 stage=done status=0。识别正文为“一位戴眼镜的年轻男子，耳朵里塞着白色耳机，正对着镜头。”

r14 另一条命令触发的实测查询采集 17,527 字节 JPEG，返回 78 字节中文结果，stage=done status=0。现有保存证据包含两次成功查询，其中一次由 KEY1 触发。

3.5.3 性能和资源数据

本次 r14 保存日志中的实机视觉请求次数为 2；JPEG 样本大小分别为 17,527 和 21,099 字节；MiMo 结果长度分别为 78 和 81 字节；两次均为 stage=done status=0。AP 镜像为 813,858 字节，CP 镜像为 920,754 字节，pair 镜像为 2,854,912 字节。Camera 配置为 640×480 JPEG；RPMmsg 文本槽为 256 字节。

表中数据来自 r14 保存的两次成功查询和构建清单，统计口径为已保存样本。

3.5.4 可靠性与异常处理

源码含 Camera 参数和 JPEG 标记校验、MiMo network/clock/credentials/http/response 阶段日志、HTTP 非 2xx、JSON 缺字段、超大结果和 RPC endpoint 未就绪处理。r7 的 DNS 失败被定位为 AP DNS 配置缺失，r8 修复后取得 HTTPS 链路；r14 补齐 KEY1 成功路径的中文正文打印。

异常处理覆盖 Camera 输入校验、网络请求失败、模型响应解析和跨核端点状态；功能结果以保存的实机日志为准。

3.6 AI-Native 开发说明

指标	数据
AI Coding 代码占比	70%（团队估算，按本队新增和改动代码的开发参与度）
使用的 AI 工具	Codex、MiMo Code、本地终端、源码检索与文档处理
MCP 工具使用情况	无
Skills 使用与新增情况	bk7258-openvela-porting；用于 BSP 分层、构建、实机验证和证据门禁，12 项测试通过
Token 使用总量	约 500,000 Token（团队估算，非平台账单统计）

3.6.1 AI 工具使用

AI Coding 用于检索 NuttX、Beken 和 OpenVela 源码，起草板级 glue 与应用接口，分析串口错误码，整理镜像清单，生成回归检查并编写技术报告。GPIO、Flash 地址、设备节点、构建结果和识别结果均由源码、VM 构建日志或 R1 串口日志复核。仓库内保存了 3 个 MiMo Code 会话、76 条事件的原始日志。

3.6.2 AI 工具指标

按本队新增和改动代码的开发参与度估计，AI Coding 代码占比为 70%；开发讨论、代码生成与调试的 Token 使用总量约为 500,000。两项均为团队估算，不是逐行归因或平台账单统计。使用工具包括 Codex、MiMo Code、本地终端、源码检索和文档处理。自建 Skill 为 bk7258-openvela-porting，项目审计 12 项全部通过。硬件实测和最终提交字段由人工确认。

3.6.3 典型问题与解决方式

Wi-Fi 初次查询在 TLS 阶段返回 -82，通过阶段日志定位为 DNS client 缺失；加入 DNS client 后重新构建并在 R1 获得 MiMo HTTP 成功。MiMo 长回答曾被固定 RPC 槽截断，通过限制提示词长度和 UTF-8 安全拷贝修正。

3.7 总结与展望

3.7.1 成果总结

本项目完成 BK7258 R1 的全 OpenVela CP/AP 架构、AP SMP、RPTUN/RPMsg、双 LCD、GC2145 V4L2 JPEG、SD 数据面和 MiMo 视觉应用闭环。r14 在真实开发板上完成 KEY1 拍照识别并打印中文结果，另有一次 NSH 命令触发的成功查询，形成面向视障场景的免手持视觉辅助原型。构建清单、镜像 SHA256、烧录日志和串口证据收录在仓库 evidence/final/。

3.7.2 应用前景与商业价值

视觉辅助胸牌面向具有独立出行能力、但在最后一段路线中仍需要视觉信息补充的视障用户，优先覆盖车站、校园、医院、商场和公共服务场所。用户可在胸牌佩戴状态下获取入口、门牌、站点标识、设施名称和简短场景描述，减少反复举起手机或求助他人的负担。当前成果验证了 Camera、LCD、网络、跨核系统和云端图片理解的产品基础，也可作为 BK7258 R1 的 OpenVela 教育与开发套件示例。

3.7.3 后续工作

围绕连续采集、网络请求和长时间佩戴运行，补充性能、稳定性、功耗与弱网数据。

接入麦克风语音提问、语音识别和回答播报，使用户能够通过“前面是什么”“入口在哪里”等自然语言完成交互。

完善 LED、振动等状态提示，并在车站、校园、医院和公共服务场所开展场景验证，同时加强隐私保护。